

AI Insight Engine — Base Platform

Is not automated, it is manually click
per-component narrative - embed in KPI, chart, table
summary (positive & negative) - collapsible section header

Automated analysis engine that reviews dashboard sections using an LLM and produces structured summary insights (positive and negative findings) plus per-component narratives. Embedded into every dashboard page via a collapsible section header.

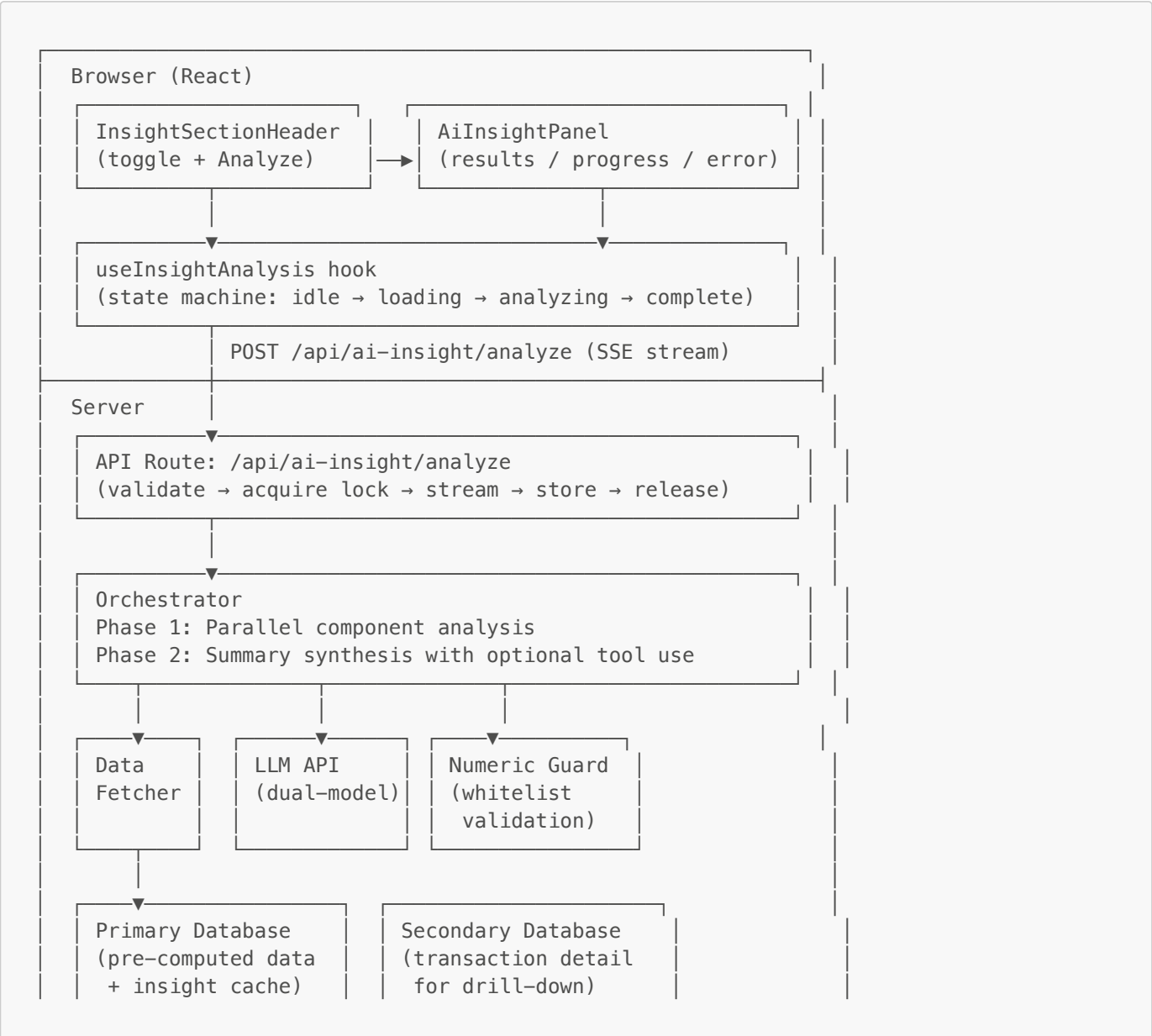
1. Purpose & User Goals

The AI Insight Engine lets senior directors get analyst-grade summaries of any dashboard section with a single click, without leaving the dashboard. It answers:

- What are the most important positive and negative findings in this section?
- What is the root cause & supporting evidence behind each finding?
- What does each individual chart, KPI, or table tell us?

The engine is domain-agnostic — it provides the analysis machinery, while domain-specific configuration (prompts, data sources, thresholds) is supplied separately.

2. Architecture Overview





3. Dual-Model Strategy

The engine uses two LLM models with different roles:

Role	Purpose	Max Tokens
Component Analysis	Fast, low-cost narration of pre-fetched data — no tool use	2,048
Summary Synthesis	Cross-component synthesis with tool-assisted root-cause investigation	4,096

Please emphasize this



Both models are configurable via environment variables. Token pricing is tracked per call (input and output) using a per-model pricing table. Cost is accumulated per section and capped.

4. Analysis Lifecycle

Analysis runs in two sequential phases.

4.1 Phase 1 — Parallel Component Analysis

1. Look up the section's component list from the section registry.
2. For each component (in parallel, up to a concurrency limit): a. **Fetch data** — call the component's data fetcher, which queries the database and returns a formatted prompt block plus an allowed-values whitelist. b. **Classify scope** — automatically label the metric as period-based, snapshot, or fiscal-period and inject a scope disclaimer into the prompt. c. **Build prompts** — combine the global system prompt, the component-specific system prompt, and the user prompt with fetched data. d. **Call the component-analysis model** — single LLM call, no tools. The model narrates and interprets the pre-fetched data. e. **Track tokens and cost** — accumulate input and output tokens. f. **Report progress** — send an SSE **progress** event per component.
3. Abort if cost exceeds the per-section cap or runtime exceeds the timeout.

Missing for Phase 2

- Fetch data
- Build prompt
- Track token & See progress

4.2 Phase 2 — Summary Synthesis

1. Build a summary prompt containing all raw data blocks from all components.
2. Call the summary model with tool access (per the section's tool policy).
3. **Agent loop** — if the model requests a tool call, execute it (up to the tool-call limit), then re-prompt. After the limit, force final output.
4. Parse the delimiter-based output format into structured positive/negative insights.
5. **Numeric guard** — extract all numbers from the output, match against the aggregated whitelist. If unmatched numbers are found, reject and re-prompt (up to 2 attempts).
6. Attach the guard report to the final output.

What else missing

Instead of parser to format the output, if we use json can it solve the same problem

Tooling

- We do have list of tools, is there rule saying, use tool sequentially, if already have enough context to give insight, stop exploring the tooling. Don't dive too deep

4.3 Storage

1. Delete any existing results for this page and section key (cascade deletes components).
2. Insert new section row with summary, metadata (time, tokens, cost, user, date scope).
3. Insert component rows with per-component narrative.
4. Release the global lock.
5. Send SSE **complete** event with section ID and metadata.

5. Concurrency & Locking

Parameter	Value	Purpose
Global Lock	One analysis at a time	Only one analysis runs at a time across all users and all sections. Simple and sufficient for a small team
Stale Lock Expiry	6 minutes 5 minutes	Auto-releases locks from crashed sessions
Parallel Component Limit	2	Parallel LLM calls per phase (avoids rate limits)
Rate Limit Retries	3 attempts, exponential backoff (15s base)	Handles 429 errors from the LLM API
Max Cost Per Section	\$0.20 USD change to 0.2	Hard cost cap per analysis run — abort if exceeded
Max Runtime	5 minutes change to 3 minutes	Timeout — aborts via controller
Max Tool Calls Per Summary	2	Summary can drill down for root causes

Lock Lifecycle

1. **POST /analyze** → acquire lock (update singleton row where unlocked or stale).
2. If lock held by another user → return 409 with lock owner info.
3. On analysis complete, error, or cancel → release lock.
4. On status check → auto-release if lock age exceeds the stale threshold (6 minutes).

6. Numeric Guard (Hallucination Prevention)

If we provide a proper output, is the numeric guard rail still necessary. This guardrail handled programmatically or using LLM

Prevents the LLM from fabricating numbers not present in the source data.

Algorithm

1. **Extract numbers** from LLM output using ordered regex patterns:
 - Currency amounts (with magnitude suffixes, ranges, comma-formatted)
 - Percentages
 - Day counts
 - Integer counts (with unit words)
2. **Strip dates** — remove date-like patterns and year tokens.
3. **Skip safe integers** — small common integers (0–12, 30, 60, 80, 90, 100, 120, 365) are always allowed.
4. **Match against whitelist** — each extracted number is checked against all allowed values from component fetchers and tool results.
 - Tolerance varies by unit type (currency ±1, percentage ±0.1, days ±0.1, count ±0.5).
 - Currency also allows: absolute-value match (for deductions), 5% relative tolerance (for rounding with magnitude suffixes).
 - Derived percentages: if a found percentage equals a ratio of two whitelisted values (±0.2), it passes.
5. **Reject and retry** — if unmatched numbers found, format an error listing them and re-prompt the LLM. Max 2 guard attempts.
6. **Report** — attach guard report (passed/attempts/unmatched) to final output.

7. Tool Use System

The tool look so little. Is this a tooling wrapper?

7.1 Tool Types

Tool	Database	Purpose	Restrictions
query_local_table	Primary (pre-computed)	Query pre-aggregated tables	Column whitelist enforced, max 100 rows, parameterized queries
query_rds_table	Secondary (transactional)	Drill into raw transaction tables	Column whitelist enforced, max 100 rows, required filters enforced

7.2 Tool Policy Tiers

Tier	Tools Available	Use Case
none	No tools	Section does not need drill-down
aggregate_only	query_local_table restricted to aggregate tables	Overview sections — aggregates only
full	Both query_local_table and query_rds_table	Breakdown sections — full drill-down

Policy is enforced per section. If the LLM requests a tool that violates the policy, the request is rejected with an error message (not silently dropped).

7.3 Security

- **Column whitelists** per table — only declared columns are queryable.
- **Row limit** — max 100 rows per query.
- **Parameterized queries** — input values are passed as parameters, never interpolated.
- **Table whitelists** — only declared tables are accessible.

8. Storage Schema

8.1 Lock Table

Column	Type	Purpose
id	Integer (PK, constrained to 1)	Singleton row — only one lock exists at a time
locked_by	Text	User who acquired the lock
locked_at	Timestamp with time zone	When lock was acquired
section_key	Text	Which section is being analyzed

8.2 Section Table

Column	Type	Purpose
id	Serial (PK)	Auto-increment ID
page	Text	Dashboard page key
section_key	Text	Section identifier
summary_json	JSONB	Structured positive/negative insights
analysis_time_s	Numeric(6,1)	Total analysis duration in seconds
token_count	Integer	Total tokens consumed
cost_usd	Numeric(8,4)	Estimated cost in USD
date_range_start	Date	Start of analyzed period (calendar scope)
date_range_end	Date	End of analyzed period (calendar scope)

Column	Type	Purpose
fiscal_year	Text	e.g. "FY2025" (fiscal scope)
fiscal_range	Text	Fiscal range type (fiscal scope)
generated_by	Text	User who triggered analysis
generated_at	Timestamp with time zone	When analysis completed

Unique constraint: (page, section_key) — one stored result per section.

8.3 Component Table

Column	Type	Purpose
id	Serial (PK)	Auto-increment ID
section_id	Integer (FK → Section, cascade delete)	Parent section
component_key	Text	Component identifier
component_type	Text	Component classification (e.g. kpi, chart, table)
analysis_md	Text	AI-generated markdown narrative
token_count	Integer	Tokens consumed for this component
generated_at	Timestamp with time zone	When analysis completed

Unique constraint: (section_id, component_key)

Upsert behavior: On re-analyze, the existing section row (and all child components via cascade) is deleted, then new rows are inserted in a transaction.

9. API Endpoints

Method	Path	Purpose	Request	Response
POST	/api/ai-insight/analyze	Start section analysis	{ page, section_key, date_range, fiscal_period, user_name }	SSE stream
GET	/api/ai-insight/status	Check global lock status	—	{ locked, locked_by, locked_at, section_key }
GET	/api/ai-insight/section/:section_key	Retrieve cached section insight	—	Section row with summary and metadata
GET	/api/ai-insight/component/:section_key/:component_key	Retrieve cached component insight	—	Component row with narrative and metadata

Method	Path	Purpose	Request	Response
POST	/api/ai-insight/cancel	Cancel running analysis	{ section_key }	{ message }

SSE Event Types

Event	Payload	When
progress	{ component, status, message? }	Per-component status update (analyzing / complete / error)
complete	{ section_id, analysis_time_s, token_count, cost_usd }	Analysis finished successfully
error	{ message }	Unrecoverable error
cancelled	{ message }	User cancelled or timeout

SSE is delivered via POST response (not EventSource) using fetch with a ReadableStream on the client.

10. UI State Machine

10.1 Hook States

The `useInsightAnalysis` hook manages analysis state.

States: `idle` → `loading` → `analyzing` → `complete` | `error` | `blocked`

State	Trigger	Description
idle	Initial / after cancel	No analysis running, no stored data
loading	On mount	Fetching stored insight from API
analyzing	User clicks "Analyze"	SSE stream active, progress lines updating
complete	SSE <code>complete</code> event	Results displayed in panel
error	SSE <code>error</code> event	Error message displayed
blocked	Lock held by another user	"Analysis running by [user]" message shown

10.2 Transitions

- **Mount** → `loading` (fetch stored) → `complete` (if exists) or `idle` (if not)
- **Analyze click** → check lock → `blocked` (if locked) or `analyzing` (SSE stream)
- **SSE complete** → re-fetch stored → `complete`
- **SSE cancelled** → `idle` + reload previous stored result
- **Cancel click** → `idle` + abort reader + server cancel + reload previous

11. UI Components

11.1 InsightSectionHeader

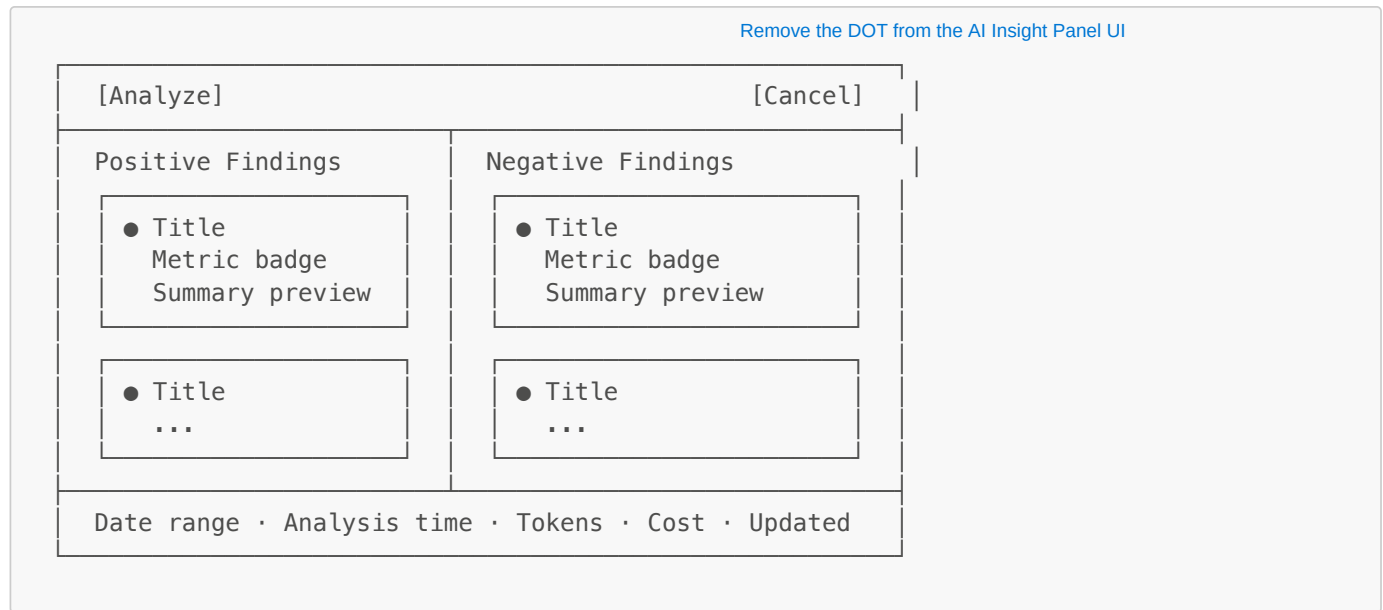
- Wraps each dashboard section with a collapsible "Get Insight" toggle.
- Contains the `AIInsightPanel` inside the collapsible region.
- The "Analyze" button is admin-only.
- Accepts: page key, section key, date range, fiscal period, user name.

11.2 AiInsightPanel

Main display panel for insight results.

Five visual states: idle (no data), loading, analyzing (progress lines), complete (two-column cards), blocked, error.

Complete state layout:



- Each insight is a card with: colored dot, title (truncated), metric badge, one-line summary preview.
- Clicking a card opens InsightDetailDialog.
- Max 3 positive + 3 negative insight cards.
- Footer: metadata strip showing date range, analysis time, token count, cost, last updated, generated by.

11.3 InsightDetailDialog

- Modal showing the full analyst report for a single insight.
- Colored header: green (positive) or red (negative).
- Scrollable body with rendered markdown.

11.4 ComponentInsightDialog

- Modal showing per-component detail.
- **About section:** static business explanation from the component-info registry.
- **AI Analysis section:** stored narrative rendered as markdown.
- Fetches component data on open from the component endpoint.

11.5 Analyzelcon

- Small icon button placed on individual dashboard charts, KPIs, and tables.
- Clicking opens ComponentInsightDialog for that specific component.

11.6 MarkdownRenderer Which is better LLM return markdown or json. Can I have table in json format

- Converts AI-generated markdown to styled HTML.
- Custom handling for: tables (scrollable, bordered), subtitle detection, consistent typography.

12. Insight Output Format

12.1 Summary Insights

Delimiter format:

```
===INSIGHT===
sentiment: good|bad
title: Short headline (max 50 chars)
metric: Key number (max 25 chars)
summary: One-line preview (max 80 chars, no markdown)
---DETAIL---
Full markdown analysis (220-320 words)
===END===
```

Parsed structure: { good: Insight[], bad: Insight[] } Each insight: { title, metric?, summary?, detail }

Constraints: Max 3 positive + 3 negative. Ranked by business impact. Fallback: JSON parse for backward compatibility.

12.2 Detail Structure (Mandatory Sections) [\(Good format, I like this, Let's keep the summary insight as clean and simple like this\)](#)

- 1. **Current Status** — headline number + business meaning + scope reference
- 2. **Key Observations** — 2-4 bullets with non-obvious patterns
- 3. **Supporting Evidence / Root Cause** — mandatory markdown table (3+ rows) of top contributors, or 3-5 specific bullets [Not markdown = but table, or ANSCII drawing explainer. Is this better](#)
- 4. **Implication** — 1-2 bullets on business consequence

12.3 Component-Level Analysis [Where is the format for component-level output](#)

- Per-component markdown narrative stored in the component table.
- Max 150 words per component.
- Generated by the component-analysis model (no tool use) from pre-fetched data.

13. Scope Types

The engine supports three scope types. Each section is assigned one scope type that governs how date context is passed to the analysis.

Scope	Behavior
period	Uses calendar date range picker (start and end date)
snapshot	Current state, not time-filtered. Anchored on the latest snapshot date from the source table.
fiscal_period	Uses fiscal year + range selector

Scope classification is injected into each component prompt so the LLM uses appropriate temporal language.

14. Debug & Observability

- **File-based debug logging** enabled via environment variable.
- Creates per-session log files with timestamped names.
- Logs: system/user prompts, API responses (token counts, cost, cache stats), tool calls and results, numeric guard attempts, session completion with totals.
- **Console prompt logging** via separate environment variable.
- Logs prompts and responses to stdout with dividers.

15. Data Protection: Column Whitelisting

The standard PII/data protection strategy across all domains is **column whitelisting at the SQL query layer**. This is the simplest approach that works for both Finance and HR.

How It Works

1. **Define allowed columns per table** — each domain declares a whitelist of columns the LLM's tools can access (e.g., `LOCAL_WHITELIST`, `RDS_WHITELIST`).
2. **Validate at query time** — `validateColumns()` rejects any tool call requesting columns not in the whitelist.
3. **Sensitive columns never appear** — emails, phone numbers, bank accounts, personal IDs are simply never added to any whitelist.

What Gets Exposed (By Design)

- **Finance:** Customer/company names, debtor/creditor codes, sales agent names, invoice numbers — needed for actionable insights ("Customer ABC owes RM 500K").
- **HR:** Department names, aggregate counts, flagged status — employee names and personal details are never whitelisted.

Domain-Specific Extensions

Each domain may add extra protection layers on top of column whitelisting:

- **HR pre-flagging:** Backend computes flagged/not-flagged status against `hr_settings` thresholds before sending to AI. The AI reads flags, not raw threshold values.
- **HR aggregate-only prompts:** Component prompts instruct the AI to analyze department-level patterns, not individuals.

What Columns to Whitelist

When adding a new domain, list every column the AI needs to see. Everything else is blocked by default. Err on the side of fewer columns — you can always add more later.

16. Deterministic Summary Questions

This should be part of the prompt builder for summary right

Each section must define a fixed set of **summary questions** that the AI answers during Phase 2 (summary synthesis). This makes the summary output predictable — same data always produces answers to the same questions.

How It Works

The summary system prompt includes a section like:

```
Answer these questions in order:
1. [Specific question about the data]
2. [Specific question about the data]
3. [Specific question about the data]
```

Guidelines for Writing Summary Questions

- **Be specific, not broad.** Bad: "Is the company healthy?" Good: "Is current ratio above 1.5?"
- **Include thresholds.** Bad: "Is attendance good?" Good: "Is monthly avg attendance above 85%?"
- **2–4 questions per section.** More than 4 makes the summary too long.
- **Questions must be answerable from the component data.** Don't ask questions that require data the AI doesn't have.

Each domain PRD (docs 11 and 12) lists the specific summary questions per section.

17. Summary System Prompt Template

Each domain must define a **summary system prompt** used during Phase 2 (summary synthesis). The prompt follows this shared structure:

Required Sections in Summary Prompt

1. **Persona** — domain-specific analyst role (e.g., "You are a senior financial analyst..." or "You are an HR analyst...")

2. **Output format** — reference to the `===INSIGHT===` delimiter format (see §12)

3. **Insight limits** — max 3 good + 3 bad insights, ranked by business impact

4. **Ground truth rule** — every number must trace to the raw data blocks or tool results. No fabrication.

5. **Sub-period citation rule** — can only cite pre-computed averages, not manually average monthly values

6. **Scope-appropriate language** — must match the section's scope type (period, snapshot, fiscal_period)

7. **Deterministic questions** — the section's fixed questions (see §16)

8. **Detail structure** — mandatory sections for the `---DETAIL---` block:

◦ **Current Status** — headline number + business meaning

◦ **Key Observations** — 2–4 bullets with non-obvious patterns

◦ **Supporting Evidence / Root Cause** — markdown table (3+ rows) or 3–5 specific bullets

◦ **Implication** — 1–2 bullets on business consequence

9. **Tool guidance** — what tools are available, what NOT to re-query (data already in raw blocks)

Ensure to add sequence of which tooling to access. If can draw conclusion, stop access tooling. No need to go deep.
- ### Domain-Specific Rules
- Each domain adds rules specific to its data:
- **Finance:** Currency format (RM with thousands separators), verbatim-copy rule, component word limit (150), summary word limit (220–320)

• **HR:** PII ban (never reference employee names/IDs), department-level language only, threshold awareness (pre-flagged data, don't re-evaluate)
- Each domain PRD (docs 11 and 12) includes the full domain-specific summary prompt text.
- ## 18. RBAC (Role-Based Access Control)
- Each domain defines its own RBAC rules. The base engine provides the implementation pattern:
- ### 18.1 Three Enforcement Layers
- | Layer | What It Does | Who Defines It |
|-------------------------|--|----------------|
| Server-side route guard | Reject unauthorized roles with 403 on <code>/analyze</code> and <code>/cancel</code> | Domain PRD |
| Data-scoping service | Inject role-based WHERE filters into every fetcher query | Domain PRD |
| Client-side UI guard | Hide/show "Analyze" button via role context hook | Domain PRD |
- ### 18.2 Cache Scoping
- When different roles see different data (e.g., HR manager sees only their department), the same section produces different results per user. The storage unique constraint must include `user_id`:
- **Finance:** All authorized roles see the same data → `UNIQUE(page, section_key)` — no user scoping needed

• **HR:** Managers see department-scoped data → `UNIQUE(module, section_key, user_id)` — per-user caching required

I can see a problem here, because we save the insight in the DB, let say HR run it where is not department specific, manager also can see. So I think this has no point. Remove this and standardize as per Finance.
- ### 18.3 What Each Domain PRD Must Define
1. **Role table** — which roles can trigger analysis, which can view insights, which are denied
- 10 / 13

2. **Data scope per role** — what data each role sees (all employees, own department, own team, etc.)
3. **Cache strategy** — whether results are shared or per-user

See each domain PRD for specific role definitions and access rules.

19. Threshold Strategy

No warning or critical, Only good and bad

Each domain defines performance thresholds that determine whether a metric is "good," "warning," or "critical." The base engine supports two approaches:

19.1 Hardcoded Thresholds (Finance Pattern)

Thresholds are embedded directly in the component prompt text and the PRD component tables.

- Simple, no database dependency
- Changed by editing prompts and redeploying
- Example: "Collection Rate $\geq 80\%$ Good, $\geq 50\%$ Warning, $< 50\%$ Critical"

19.2 Configurable Thresholds (HR Pattern)

Thresholds are stored in a settings table and loaded at analysis time. Fetchers pre-flag data against these thresholds before sending to the AI.

- Admin-configurable via Settings UI without code changes
- AI sees "flagged / not-flagged" — does not re-evaluate raw numbers against thresholds
- Example: `alert_chronic_lateness.threshold = 3` → fetcher flags employees with ≥ 3 late days

19.3 Pre-Flagging Pattern

When using configurable thresholds, the fetcher (not the AI) evaluates data against thresholds:

1. **Settings loader** batch-fetches all threshold categories from the settings table
2. **Fetcher** applies thresholds to compute flags (e.g., "flagged_chronic_lateness: true")
3. **AI prompt** receives flagged results — instructed not to re-evaluate
4. **Frontend** also displays flags in the dashboard UI (dual use)

Each domain PRD lists its specific thresholds and whether they are hardcoded or configurable.

20. Extensibility Contract

The base engine is domain-agnostic. To add a new domain, provide:

1. **Section Registry** — map section keys to component lists.
2. **Section Names** — display names for each section.
3. **Section-to-Page Mapping** — map sections to dashboard pages.
4. **Component Prompts** — per-component system prompts with domain-specific analysis instructions.
5. **Component Info** — static "About" content for each component.
6. **Data Fetchers** — per-component functions that query the domain's database and return formatted prompt blocks + allowed-value whitelists.
7. **Tool Policy** — per-section tool access tier (none, aggregate_only, full).
8. **Global System Prompt** — domain-specific persona and rules (currency, terminology, thresholds).
9. **Summary System Prompt** — domain-specific output format rules and table/column references.

What stays shared: Orchestrator, numeric guard, lock, storage, API routes, SSE protocol, UI components, debug logger, tool execution engine.

21. Implementation Guardrails

These patterns were discovered during implementation and validated through accuracy testing. They prevent specific classes of bugs — removing any of them reintroduces known failures.

21.1 Pre-Calculated Totals in Fetchers

Data fetchers that pass monthly breakdowns must also include pre-calculated totals, ratios, and percentages in the prompt output. The component-analysis model (Haiku) cannot do arithmetic reliably — it summed 12 monthly values incorrectly (RM 87M vs actual RM 82M) and hallucinated percentages (claimed 95% when actual was 58%).

All fetchers include a `"Pre-calculated roll-ups (use these values directly – do not recompute)"` block above their data tables.

21.2 Tool Call Exhaustion Nudge

When the summary uses all allowed tool calls, the orchestrator injects a user message:

```
"You have used all available tool calls. Now produce your final summary using the ===INSIGHT=== delimiter format.
Do not request more data — work with what you have."
```

Without this nudge, the model outputs reasoning text instead of the required delimiter format, causing the parser to fall back to a generic output.

21.3 Snapshot Table Deduplication

The `pc_ar_customer_snapshot` table contains multiple rows per customer (one per snapshot date). Tool queries against this table are automatically deduplicated by the tool executor: filter to latest `snapshot_date` and apply `DISTINCT ON (debtor_code)`.

Without deduplication, a "top 10 customers" query returns duplicate rows of the same customers, making root-cause analysis useless.

21.4 Summary Tool Call Guidance

The summary system prompt explicitly instructs the LLM not to re-query data already available in the raw data blocks. Without this guidance, the summary wastes both tool calls re-querying monthly data already available, leaving none for actual root-cause investigation.

21.5 Summary Reads Raw Fetcher Data, Not Narrations

The summary stage receives the raw fetcher markdown blocks (the same formatted string each component originally saw) — NOT the Haiku component narrations. The `ComponentResult` type carries a `raw_data_md` field populated from the fetcher output. The summary system prompt enforces a GROUND TRUTH RULE: every number must be traceable to a specific line in the raw blocks or to a tool-call result.

Without this, the summary model invents numbers to fill tables because narrations (~150 words) don't contain source values. This single fix eliminated all hard fabrications across ~115 numeric claims.

21.6 Population Labels

Each data fetcher includes a population label in its output header describing which records are included (e.g., "Population: active customers only (is_active = 'T')"). This prevents the LLM from cross-referencing numbers between components that use different populations and getting confused by the mismatch.

21.7 Active Customer Filtering

Fetchers that query `pc_ar_customer_snapshot` apply different `is_active` filters depending on the query purpose:

Query Purpose	Filter
Credit Limit Breaches	<code>is_active = 'T'</code> (only active breaches are operationally relevant)
Credit Usage Distribution	<code>is_active = 'T' OR is_active IS NULL</code> (match dashboard donut)
Total Outstanding / Overdue	No filter (inactive customers still owe money)
Customer Credit Health table	<code>is_active = 'T' OR total_outstanding > 0</code> (active + any inactive that still owe)

22. Known Limitations & Gaps

22.1 Implemented Behaviors

Item	Detail	Impact
Error handling	Cost limit abort (\$0.20/section), timeout auto-cancel (5 min), rate limit retry (3 attempts, 15s exponential backoff)	Medium
Navigation limitation	SSE disconnects on page change; stale lock auto-releases after 6 minutes. Users should stay on the page during analysis.	Low
Data-fetcher date format	<code>toMonth()</code> helper converts <code>YYYY-MM-DD</code> to <code>YYYY-MM</code> for <code>pc_ar_monthly</code> queries. Without this, string comparison silently excludes the first month.	Low
Blocked PII columns	<code>attention</code> , <code>phone1</code> , <code>mobile</code> , <code>email_address</code> are not in column whitelists — implicitly blocked from AI access.	Medium

22.2 Required — Not Yet Implemented

These safety rules must be enforced in code before HR goes live:

- **Server-side `Cancelled = 'F'` injection** — currently prompt-level only. The tool description tells the LLM to include the filter, but `executeRdsQuery()` does not inject it server-side if the LLM omits it. **Fix:** Inject `Cancelled = 'F'` server-side in `executeRdsQuery()` for all RDS tables that have the column.
- **SQL injection guard** — `where_clause` from the LLM is passed directly into SQL without pattern checking. **Fix:** Reject WHERE clauses containing `;`, `UNION`, `DROP`, `DELETE`, `UPDATE`, `INSERT`, `--`, or subqueries before execution.